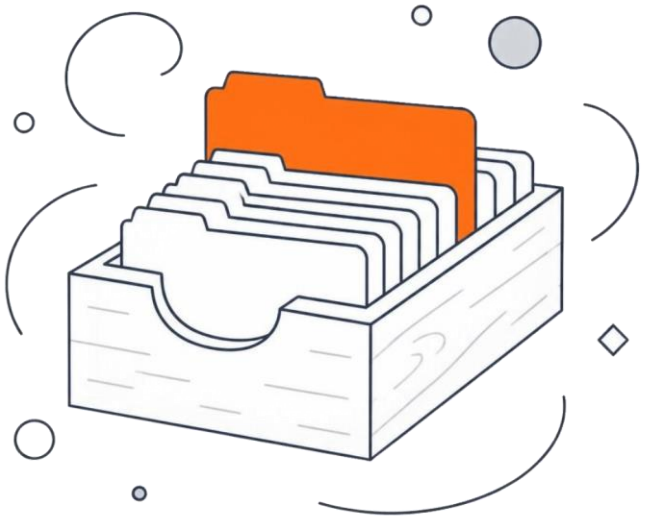
Язык программирования Python · Списки

Тема урока: вывод элементов списка

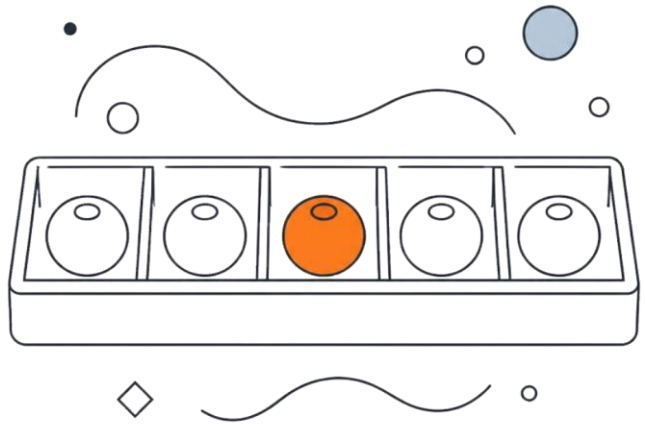
1. Вывод списка с помощью `for`
2. Вывод списка с помощью распаковки
3. Вывод строки с помощью распаковки



Аннотация

Вывод элементов списка и строки.

При выводе содержимого списка с помощью функции `print()` вывод элементов осуществляется в квадратных скобках, причём все элементы разделены запятой. Такой вывод не всегда удобен и предпочтителен, поэтому нужно уметь выводить элементы списка нужным нам способом.



Вывод с помощью цикла for

Для вывода элементов списка **каждого на отдельной строке** можно использовать следующий код:

Вариант 1. Если нужны индексы элементов:

by_index.py

```
numbers = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
for i in range(len(numbers)):
    print(numbers[i])
```

Вывод в консоль

```
0
1
2
...
10
```

Вывод с помощью цикла for

Мы передаем в функцию `range()` длину списка `len(numbers)`. В нашем случае длина списка `numbers` равна `11`. Таким образом, вызов функции `range(len(numbers))` имеет вид `range(11)`, и переменная цикла `i` последовательно перебирает все значения от `0` до `10`. Это означает, что выражение `numbers[i]` последовательно вернёт все элементы списка `numbers`. Такой способ итерации списка удобен, когда нам нужен не только сам элемент `numbers[i]`, но и его индекс `i`.

Параметр	Значение
`len(numbers)`	`11`
`range(11)`	`0..10`
Переменная `i`	индекс
`numbers[i]`	элемент

Вывод с помощью цикла for

Вариант 2. Если индексы не нужны:

by_value.py

```
numbers = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
for num in numbers:
    print(num)
```

Вывод в консоль

```
0
1
2
...
10
```

Этот цикл пройдёт по списку `numbers`, придавая переменной цикла `num` значение каждого элемента списка (!) в отличие от предыдущего цикла, в котором переменная цикла «бегала» по индексам списка.

Вывод с помощью цикла for

Если требуется выводить элементы списка на одной строке через пробел, то мы можем использовать необязательный параметр `end` команды `print()`:

print_end.py

```
numbers = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
for num in numbers:
    print(num, end=' ')
```

Вывод в консоль

```
0 1 2 3 4 5 6 7 8 9 10
```

Вывод с помощью распаковки списка

В Python есть удобный способ вывода элементов списка без использования цикла `for`.

Вариант 1. Вывод элементов списка через один символ пробела:

unpack_space.py

```
numbers = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
print(*numbers)
```

Приведённый выше код выводит:

```
0 1 2 3 4 5 6 7 8 9 10
```

Вывод с помощью распаковки списка

Вариант 2. Вывод элементов списка, каждого на отдельной строке:

unpack_sep.py

```
numbers = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
print(*numbers, sep='\n')
```

Приведённый выше код выводит:

```
0
1
2
3
4
5
6
7
8
9
10
```

Вывод строки с помощью распаковки

Поскольку строки содержат символы, подобно тому, как списки содержат элементы, то мы можем использовать распаковку строк точно так же, как и распаковку списков.

Приведённый ниже код:

unpack_str.py

```
s = 'Python'
print(*s)
print()
print(*s, sep='\n')
```

выводит

```
P y t h o n

P
y
t
h
o
n
```



Примечания

Примечание 1. Пусть у нас в программе определён список:

```
date = [31, 'января', 1956]
```

Распаковка элементов списка при выводе происходит так, словно мы вручную записали все его элементы в команду `print()` через запятую. Следующие строки кода являются эквивалентными:

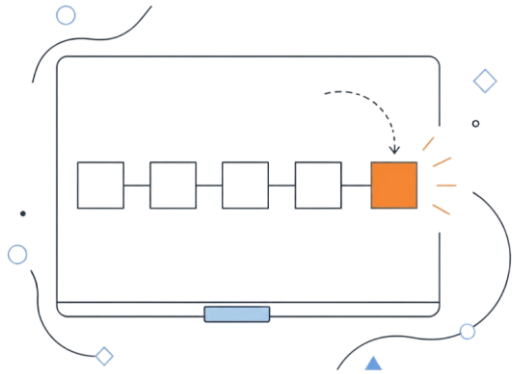
```
print(*date)
print(date[0], date[1], date[2])
```

и выводят:

```
31 января 1956
```



Добавление элементов



Мы научились создавать статические списки, то есть списки, элементы которых известны на этапе создания. Следующий шаг – научиться добавлять элементы в уже существующие списки.

Метод append()

Для добавления нового элемента в конец списка используется метод append().

Приведённый ниже код:

```
numbers = [1, 1, 2, 3, 5, 8, 13]  # создаем
список

numbers.append(21)  # добавляем число 21 в конец
списка
numbers.append(34)  # добавляем число 34 в конец
списка

print(numbers)
```

выводит:

```
[1, 1, 2, 3, 5, 8, 13, 21, 34]
```

Метод append()

Обратите внимание, для того чтобы использовать метод append(), нужно, чтобы список был создан (при этом он может быть пустым).

Приведённый ниже код:

```
numbers = []  # создаем пустой список

numbers.append(1)
numbers.append(2)
numbers.append(3)

print(numbers)
```

выводит:

```
[1, 2, 3]
```

Метод append()

Важно: мы не можем использовать индексы для установки значений элементов списка, если список пустой.

Приведённый ниже код:

```
numbers = []  # создаем пустой список

numbers[0] = 1
numbers[1] = 2
numbers[2] = 3

print(numbers)
```

приводит к возникновению ошибки:

```
IndexError: list assignment index out of
range
```

Метод extend()

Можно также расширить список другим списком путём вызова метода extend().

Приведённый ниже код:

```
numbers = [0, 2, 4, 6, 8, 10]
odds = [1, 3, 5, 7]

numbers.extend(odds)
print(numbers)
```

выводит:

```
[0, 2, 4, 6, 8, 10, 1, 3, 5, 7]
```

Метод extend() как бы расширяет один список, добавляя к нему элементы другого списка.

Метод extend()

Отличие между методами append() и extend() проявляется при добавлении строки к списку.

Приведённый ниже код:

```
words1 = ['iq option', 'stepik', 'beegeek']
words2 = ['iq option', 'stepik', 'beegeek']

words1.append('python')
words2.extend('python')

print(words1)
print(words2)
```

выводит:

```
['iq option', 'stepik', 'beegeek',
'python']
['iq option', 'stepik', 'beegeek', 'p',
'y', 't', 'h', 'o', 'n']
```

Метод append() добавляет строку 'python' целиком к списку, а метод extend() разбивает строку 'python' на символы 'p', 'y', 't', 'h', 'o', 'n' и их добавляет в качестве элементов списка.

Удаление элементов

С помощью оператора del можно удалять элементы списка по определенному индексу.

Приведённый ниже код:

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9]
del numbers[5]    # удаляем элемент, имеющий
индекс 5

print(numbers)
```

выводит:

```
[1, 2, 3, 4, 5, 7, 8, 9]
```

Элемент под указанным индексом удаляется, а список перестраивается.

Обратите внимание на синтаксис удаления, так как он отличается от обычного вызова метода. При удалении элементов не надо передавать аргумент внутри круглых скобок.

Удаление элементов

Оператор del работает и со срезами: мы можем удалить целый диапазон элементов списка.

Приведённый ниже код:

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9]
del numbers[2:7]    # удаляем элементы с 2 по 6
включительно

print(numbers)
```

выводит:

```
[1, 2, 8, 9]
```

Удаление элементов

Мы можем удалить все элементы на чётных позициях (0, 2, 4, ...) исходного списка.

Приведённый ниже код:

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9]
del numbers[::2]

print(numbers)
```

выводит:

```
[2, 4, 6, 8]
```

Примечания

Примечание 1. В отличие от строковых методов, которые не изменяют исходную строку, а возвращают новую, многие списочные методы изменяют исходный список и возвращают значение None.

Приведенный ниже код:

```
animals = ['cat', 'dog', 'snake']
new_animals = animals.append('bird')

print(animals)
print(new_animals)
```

выводит:

```
['cat', 'dog', 'snake', 'bird']
None
```

В Python None – это особое значение, символизирующее "ничего". Подробнее о None мы будем говорить в курсе для продвинутых.

Примечания

Примечание 2. Обратите внимание: для изменения элементов списка необходимо явно обращаться по индексу к этому списку.

Приведённый ниже код:

```
numbers = [4, 7, -2, 1, 6]

for num in numbers:
    num = num * 2

print(numbers)
```

выводит:

```
[4, 7, -2, 1, 6]
```

Заметим, что список не изменился: мы лишь создали на каждой итерации новую переменную с таким же именем num. Чтобы изменить значение в списке, нужно обращаться к списку явно по индексу.

Примечания

Приведённый ниже код:

```
numbers = [4, 7, -2, 1, 6]

for i in range(len(numbers)):
    numbers[i] = numbers[i] * 2

print(numbers)
```

выводит:

```
[8, 14, -4, 2, 12]
```